

Mathematical Model: Agent Political Simulation

(generated from codebase)

1 Scope and State

This document models the discrete-time simulation implemented in:

- `src/simulation/engine.py` (turn loop)
- `src/actions/decision.py` (action choice)
- `src/actions/voting.py` (voting and veto)
- `src/simulation/consequences.py` (state updates)
- `src/simulation/drift.py` (ideology drift and IG radicalization)
- `src/simulation/elections.py`, `src/simulation/factions.py` (elections/splits)

1.1 Sets and indices

Let:

$t \in \{0, 1, 2, \dots\}$	turn index
$\mathcal{P}$	set of places (jurisdictions)
$\mathcal{K}$	set of parties
$\mathcal{G}$	set of interest groups
$\mathcal{A}_p(t) \subseteq \mathcal{A}(t)$	agents currently in place p
$\mathcal{L}_p(t) \subseteq \mathcal{A}_p(t)$	legislators in place p
$\mathcal{X}$	set of ideology axes (e. g. economic, social)

1.2 World state variables

For each agent $a \in \mathcal{A}(t)$:

$\mathbf{i}_a(t) \in [-1, 1]^{ \mathcal{X} }$	ideology vector
$k(a, t) \in \mathcal{K}$	party membership
$p(a, t) \in \mathcal{P}$	place membership
$\alpha_{a,g}(t) \in [0, 1]$	allegiance weight to IG g
$r_{a,b}(t) \in [-1, 1]$	relationship/trust (directed)
$\pi_{a,g}(t) \in [0, 1]$	popularity with IG g
$s_a(t) \in [0, 1]$	party standing (loyalty)
$\eta_a(t) \in [0, 1]$	ambition (noise scale)
$\text{arch}(a) \in \{\text{LOYALIST}, \text{OPPORTUNIST}, \text{IDEOLOGUE}\}$	voting archetype

For each interest group $g \in \mathcal{G}$ and place $p \in \mathcal{P}$:

$\sigma_{g,p}(t) \in [0, 1]$	satisfaction
$e_{g,p}(t) \in [0, 1]$	electorate share (mobilized strength)
$\mathbf{i}_g(t) \in [-1, 1]^{ \mathcal{X} }$	IG ideology vector

For each party $k \in \mathcal{K}$:

$\mathbf{i}_k(t) \in [-1, 1]^{ \mathcal{X} }$	party ideology vector
$\rho_{k,k'}(t) \in [-1, 1]$	party-to-party relation
$\rho_{k,g}^{IG}(t) \in [0, 1]$	party-to-IG relation
$\theta_k \in [0, 1]$	directive threshold (party-line activation)
$B_k(t) \in \mathbb{Z}_{\geq 0}$	campaign budget (elections)

1.3 Policy objects (ephemeral)

During each turn, each government generates a pool of candidate policies $\mathcal{M}_p(t)$ (not persisted in World; see `src/models/policy.py`). For each policy m :

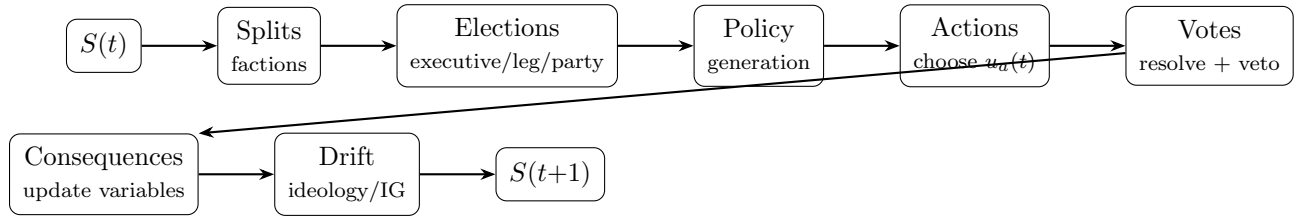
$\Delta_{m,g} \in [-1, 1]$	impact on IG g
$\mathbf{d}_m \in \mathbb{R}^{ \mathcal{X} }$	ideology alignment directions by axis

2 Turn structure as a stochastic state transition

The simulation is a discrete-time stochastic dynamical system:

$$S(t+1) = F(S(t), \xi_t),$$

where $S(t)$ is the full world state and ξ_t is exogenous randomness (the RNG draws).



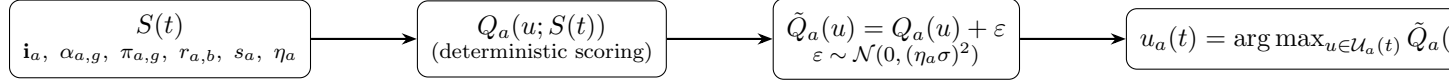
In code (`src/simulation/engine.py`), a turn is:

1. splits and elections (may change roster/roles)
2. per-place policy generation
3. per-agent action selection
4. legislative votes + executive veto + possible override
5. apply consequences of non-vote actions
6. ideological drift and IG radicalization/moderation

3 Action selection model

3.1 Action set

For each agent a , an availability filter defines a feasible set of actions $\mathcal{U}_a(t)$ (see `src/actions/availability.py`). The decision policy scores candidates and selects the maximum after adding ambition-scaled Gaussian noise (`src/actions/decision.py`).



3.2 Utility weights

Each agent has a weight vector

$$\mathbf{w}_a = (w_a^{pop}, w_a^{stand}, w_a^{ideo}, w_a^{ig}, w_a^{rel}), \quad \sum_i w_i = 1,$$

defaulting to `DEFAULT_WEIGHTS` in `src/actions/utility.py`.

3.3 Noisy argmax decision rule

For each candidate action $u \in \mathcal{U}_a(t)$ the code forms a deterministic score $Q_a(u; S(t))$, then adds noise:

$$\tilde{Q}_a(u) = Q_a(u; S(t)) + \varepsilon, \quad \varepsilon \sim \mathcal{N}(0, (\eta_a(t) \sigma_u)^2),$$

with $\sigma_u \in \{0.02, 0.01\}$ depending on (agent vs party-leader) action. Then:

$$u_a(t) = \arg \max_{u \in \mathcal{U}_a(t)} \tilde{Q}_a(u).$$

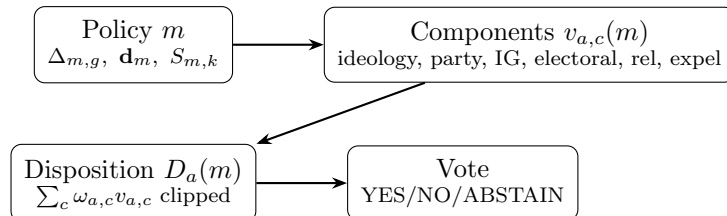
4 Voting model

4.1 Vote disposition

When a policy m is put to a vote, each legislator $a \in \mathcal{L}_p(t)$ computes a disposition score $D_a(m) \in [-1, 1]$ (see `compute_vote_disposition` in `src/actions/voting.py`):

$$D_a(m) = \text{clip}_{[-1,1]} \left(\sum_{c \in \mathcal{C}} \omega_{a,c} v_{a,c}(m) \right),$$

where components $\mathcal{C} = \{\text{ideology, party_directive, ig_pressure, electoral, relationships, expulsion_risk}\}$, with archetype-conditioned weights $\omega_{a,c}$ (possibly perturbed once per agent and stored).



4.2 Component definitions (code-faithful)

Ideology alignment. If $\mathbf{d}_m$ is the policy's axis-direction map and $\mathbf{i}_a$ agent ideology:

$$v_{a,\text{ideo}}(m) = \frac{1}{|\mathcal{X}_m|} \sum_{x \in \mathcal{X}_m} i_{a,x} d_{m,x}.$$

Party directive pressure. Policies may have an explicit party support scalar $S_{m,k} \in [-1, 1]$ stored as `policy.attributes["party.support"] [party]`. Given party threshold θ_k :

$$v_{a,\text{party}}(m) = \begin{cases} \frac{S_{m,k} - \theta_k}{1 - \theta_k} & S_{m,k} > \theta_k \\ \frac{S_{m,k} + \theta_k}{1 + \theta_k} & S_{m,k} < -\theta_k \\ 0 & \text{otherwise.} \end{cases}$$

IG pressure. Let $\Delta_{m,g}$ be impact and $\alpha_{a,g}$ allegiance. The code dampens low allegiance:

$$\alpha'_{a,g} = \begin{cases} \alpha_{a,g} & \alpha_{a,g} \geq 0.15 \\ 0.4\alpha_{a,g} & \alpha_{a,g} < 0.15 \end{cases}$$

and then

$$v_{a,\text{ig}}(m) = \frac{\sum_g \Delta_{m,g} \alpha'_{a,g}}{\sum_g \alpha'_{a,g}} \quad (0 \text{ if denominator is } 0).$$

Electoral. Popularity-weighted impact:

$$v_{a,\text{elect}}(m) = \frac{\sum_g \Delta_{m,g} \pi_{a,g}}{\sum_g \pi_{a,g}} \quad (0 \text{ if denominator is } 0).$$

Relationships.

$$v_{a,\text{rel}}(m) = r_{a,\text{proposer}(m)}.$$

Expulsion risk. If directive pressure is $v_{a,\text{party}}(m) \neq 0$, vulnerability in the code is:

$$\nu_a = 0.45(1 - s_a) + 0.35(1 - \bar{\pi}_a) + 0.20(1 - \text{altAccess}_a),$$

clipped to $[0, 1]$, and then

$$v_{a,\text{expel}}(m) = v_{a,\text{party}}(m) \cdot \nu_a.$$

(Here $\bar{\pi}_a$ is electorate-share-weighted popularity survivability, and altAccess_a depends on ideological distance to other parties as in code.)

4.3 Vote resolution, veto, and override

Each legislator casts:

$$\text{YES if } D_a(m) > 0.1, \quad \text{NO if } D_a(m) < -0.1,$$

else a gray-zone rule that can abstain when personal lean is negative but party pull is positive.

Define indicator functions for each legislator $a \in \mathcal{L}_p(t)$:

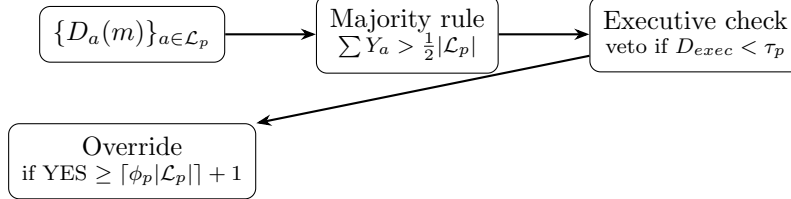
$$Y_a(m) = \mathbb{I}[D_a(m) > 0.1], \quad N_a(m) = \mathbb{I}[D_a(m) < -0.1].$$

The bill passes if:

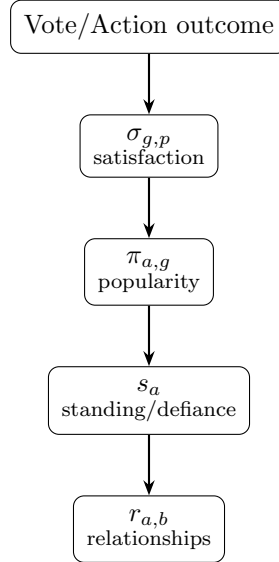
$$\sum_{a \in \mathcal{L}_p(t)} Y_a(m) > \frac{1}{2} |\mathcal{L}_p(t)|.$$

If tied (YES=NO) and an executive exists, executive breaks tie in favor iff their $D_{exec}(m) > 0$.

If passed, executive vetoes iff $D_{exec}(m) < \tau_p$ where τ_p is `gov.veto_threshold` (default -0.3). Veto override is checked later using a fraction ϕ_p (default $2/3$).



5 Consequences (state updates)



5.1 Interest-group satisfaction update

On passage, for each impacted group with satisfaction defined in the place:

$$\sigma_{g,p} \leftarrow \text{clip}_{[0,1]} (\sigma_{g,p} + \Delta_{m,g} \cdot c(\Delta_{m,g})),$$

where in code $c = 0.5$ for $\Delta \geq 0$ and $c = 0.6$ for $\Delta < 0$ (negatives hit harder).

There is also a cross-IG conflict penalty for pairs of groups with high ideological conflict.

5.2 Agent popularity update (memory of vote)

For each voter a and IG g :

$$\pi_{a,g} \leftarrow \text{clip}_{[0,1]} (\pi_{a,g} + \text{dir}(a) \Delta_{m,g} \cdot 0.18),$$

where $\text{dir}(a) = +1$ if voted YES else -1 .

5.3 Party standing update

If a party directive is active, following it increases standing; defying decreases it:

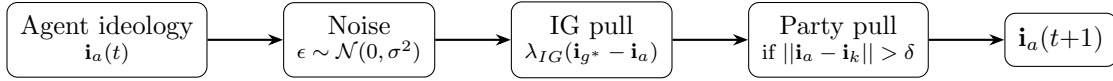
$$s_a \leftarrow \text{clip}_{[0,1]}(s_a \pm (0.01 + 0.03|v_{a,\text{party}}|) \text{ or } (0.02 + 0.08|v_{a,\text{party}}|)).$$

The code also tracks `recent_defiance`.

5.4 Relationship update

Voting together increases mutual trust (+0.02); opposing votes decrease trust (-0.01). Non-vote actions also adjust relationships and standing (see `src/simulation/consequences.py`).

6 Ideological drift and IG radicalization



6.1 Agent ideology drift

For each low-detail-level agent and axis x (see `src/simulation/drift.py`):

$$i_{a,x} \leftarrow \text{clip}_{[-1,1]}(i_{a,x} + \epsilon_{a,x} + \lambda_{IG}(i_{g^*,x} - i_{a,x}) + \mathbb{I}[|i_{a,x} - i_{k,x}| > \delta] \lambda_{party}(i_{k,x} - i_{a,x})),$$

with $\epsilon_{a,x} \sim \mathcal{N}(0, \sigma^2)$.

6.2 IG radicalization / moderation

Let $\bar{\sigma}_g$ be average satisfaction across places. If $\bar{\sigma}_g < \sigma_{rad}$, the IG becomes more extreme and more mobilized:

$$i_{g,x} \leftarrow \text{clip}(i_{g,x} \cdot \gamma + \text{sign}(i_{g,x}) \cdot 0.005), \quad e_{g,p} \leftarrow \min(1, e_{g,p} \cdot \kappa).$$

If $\bar{\sigma}_g > \sigma_{cent}$, it moderates and slightly demobilizes.

7 Tunable Variables Table (code-extracted)

The following are the primary *tweak points* exposed as fields or constants in the code. Some are hard-coded constants; others live in dataclasses and can be changed at initialization time.

Variable	Default	Effect
t	0	Turn counter.
$\mathbf{i}_a$	—	Ideological position per axis in $[-1, 1]$.
$\alpha_{a,g}$	{}	IG influence weights on agent actions/votes.
$\pi_{a,g}$	{}	IG-specific approval that affects elections/vulnerability.
$r_{a,b}$	{}	Inter-agent trust used in votes and actions.
s_a	0.5	Loyalty/standing inside party; used in voting/expulsion risk.

Variable	Default	Effect
η_a	0.5	Noise scale for action choice; affects randomness.
$\mathbf{w}_a$	DEFAULT_WEIGHTS	Per-agent action utility coefficients.
$\omega_{a,c}$	(generated)	Per-agent vote component coefficients.
θ_k	0.15	Party-line activation threshold for directives.
B_k	3	Election nomination budget decrementing on use.
—	0.3	Nomination threshold.
—	20	Leadership election interval.
τ_p	-0.3	Executive veto if exec disposition below threshold.
ϕ_p	2/3	Fraction of seats needed (plus 1) to override veto.
—	—	Per-government election schedule interval.
—	(0.3,0.2,0.15,0.2,0.15)	Default utility weights DEFAULT_WEIGHTS.
—	varies	Archetype vote weights ARCHETYPE_WEIGHTS.
—	± 0.1	Vote thresholds for YES/NO/abstain logic.
σ	0.01	Std-dev of per-turn ideology Brownian noise.
λ_{IG}	0.02	Pull rate toward top-allegiance IG ideology.
δ	0.4	Divergence threshold before party-pull activates.
λ_{party}	0.015	Pull rate back toward party ideology.
σ_{rad}	0.25	Avg satisfaction below triggers IG radicalization.
γ	1.05	Per-turn multiplier pushing IG ideology away from zero.
κ	1.02	Per-turn multiplier increasing IG electorate share.
σ_{cent}	0.72	Avg satisfaction above triggers IG moderation.
—	0.03	Centrist pull rate.
—	0.99	Centrist share decay.
—	2	Executive term limit.
—	4	Legislative term limit.
—	10	Stale-agent removal threshold.
—	0.25	Support dilution factor.
—	0.6	Viability ratio.
—	0.45	Split satisfaction threshold.
—	3	Split inertia.
—	0.10	Split ideology gap.
—	2	Minimum splinter size.
—	2	Splinter start budget.
—	0.5	Ambition floor.

Math $\leftrightarrow$ code mapping

Math variable	Code
t	World.turn
$\mathbf{i}_a$	Agent.ideology.axes
$\mathbf{i}_k$	Party.ideology.axes
$\mathbf{i}_g$	InterestGroup.ideology.axes
$\alpha_{a,g}$	Agent.allegiances[InterestGroup]

Math variable	Code
$\pi_{a,g}$	<code>Agent.popularity[InterestGroup]</code>
$r_{a,b}$	<code>Agent.relationships[Agent]</code>
s_a	<code>Agent.party_standing</code>
η_a	<code>Agent.ambition</code>
$\mathbf{w}_a$	<code>Agent.attributes["utility_weights"]</code>
$\omega_{a,c}$	<code>Agent.attributes["vote_weights"]</code>
$\sigma_{g,p}$	<code>InterestGroup.satisfaction[Place]</code>
$e_{g,p}$	<code>InterestGroup.electorate_share[Place]</code>
θ_k	<code>Party.directive_threshold</code>
B_k	<code>Party.campaign_budget</code>
τ_p	<code>Government.veto_threshold</code>
ϕ_p	<code>Government.veto_override_fraction</code>
σ	<code>_RANDOM_DRIFT_SIGMA</code>
λ_{IG}	<code>_IG_PULL_RATE</code>
δ	<code>_PARTY_PULL_THRESHOLD</code>
λ_{party}	<code>_PARTY_PULL_RATE</code>
σ_{rad}	<code>_RAD_THRESHOLD</code>
γ	<code>_RAD_AXIS_SCALE</code>
κ	<code>_RAD_SHARE_BOOST</code>
σ_{cent}	<code>_CENTRIST_THRESHOLD</code>